

SQL

This document contains an introduction to Relational Databases using SQL

Note: The terminology included in this document and a basic understanding of SQL is course content and therefore may be included on exams or quizzes.

Table of Contents

Introduction	4
Terminology	4
Limitations	4
How to follow along	4
From the command line	4
From MySQL Workbench	5
What is a database?	7
Building a database	8
Creating Tables	9
Column Modifiers	9
Primary Key	10
Foreign Keys	10
Table Constraints	12
Modifying a Database	13
Insert	13
Update	13
Delete	14
Basic Queries	14
SELECT	14
Round	14
WHERE	15
LIKE	15
ORDER BY	16
LIMIT	16
OFFSET	16
DISTINCT	17
GROUP BY	17
HAVING	17
JOIN	18
CROSS JOIN	18
INNER JOIN	19
OUTER JOINS	19
NATURAL JOIN	21
Nested Queries	22
WITH and AS	22
Union	23

Introduction

SQL stands for structured query language, a language that allows us to manipulate and communicate with relational databases. It is one of the most popular languages for relational database management systems that allows us to easily store and manipulate tabular data. Using SQL, you can insert, update, remove, and query data from databases, as well as design and structure the databases themselves.

Terminology

Schema - a collection of related tables and constructs

Database - a systematic collection of data that can support both manipulation and storage of data

Table - a collection of related data in a database that is comprised of rows and columns

SQL Data Types - primarily split into numeric, string, and temporal data types. Includes int, float/double, decimal, char, varchar, text, enum, date, datetime, timestamp, time

Constraints - rules used to limit the type of data that can go into the table. Maintains accuracy and consistency of the data within the table

Primary Key - a field in the table which uniquely identifies each row/record in a database table

Foreign Key - a field in one table that refers to a unique field in another table. Used to establish relational integrity between more than one table

Query - a request for specific data from one or more database tables

Limitations

There are entire courses and majors that teach databases, the purpose of this document is to briefly introduce the concepts and use cases of relational databases using MySQL.

How to follow along

Download the [university schema.sql](#) and [university data.sql](#) file and place the files in a subdirectory of the CS2316 folder.

From the command line

Navigate to the files location via the command line and execute the following and enter your MySQL password when prompted.

```
C:/.../CS2316/.../> mysql -u root -p < university_schema.sql
```

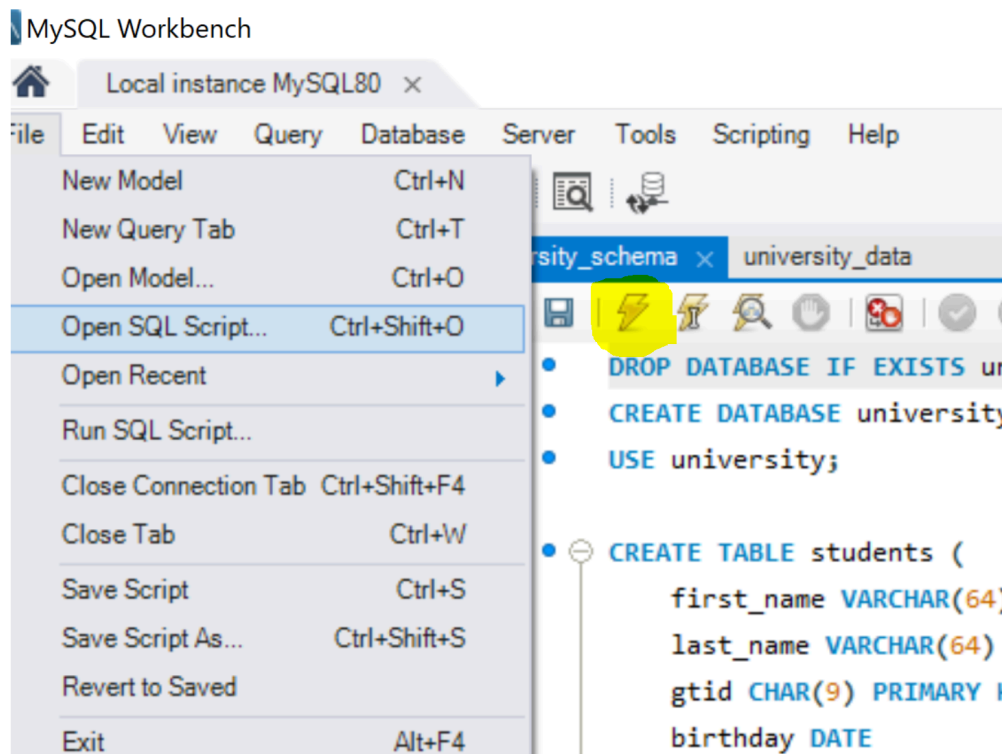
```
C:/.../CS2316/.../> mysql -u root -p < university_data.sql
```

Then to enter the MySQL shell execute

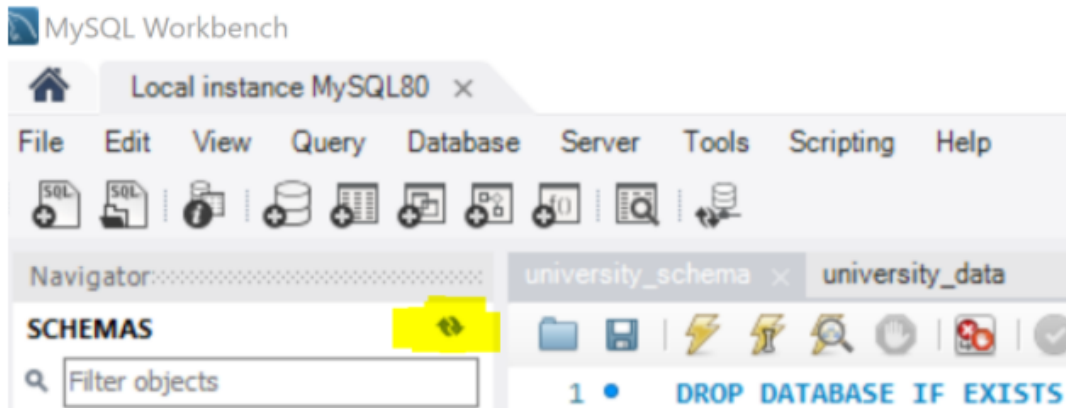
```
C:/.../CS2316/.../> mysql -u root -p
mysql> use university;
mysql> show tables;
mysql> SELECT * FROM students;
```

From MySQL Workbench

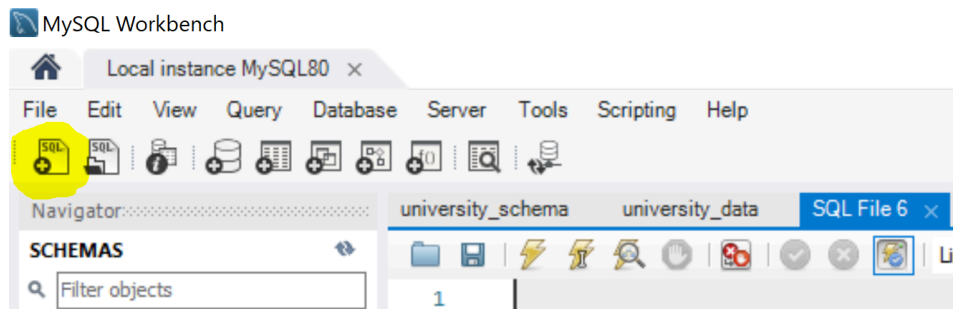
Open MySQL Workbench and select a host under the MySQL connections and enter MySQL password. Note the MySQL Workbench password may not be the same as your computer password. If you forgot your password, revisit the getting started handout or come to office hours. After installing the `university_schema.sql` and `university_data.sql` files, in MySQL workbench open both files by selecting file then Open SQL Script.



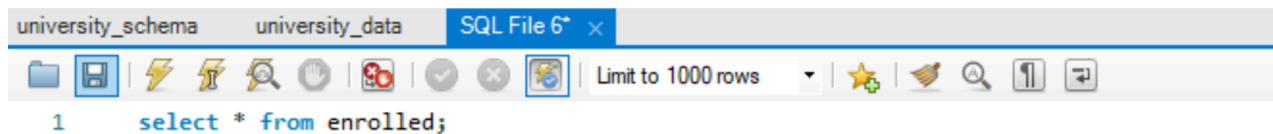
After opening each file, click the lightning bolt (highlighted in yellow above) to execute the script. To access the university database, *make sure* to click the refresh button in the schema navigator, as shown in the screenshot below.



To execute queries, you must create a new SQL file by selecting the button highlighted below.



In the new file, type the following query and click the lightning bolt to see the resulting table. Note that the leftmost lightning bolt will execute all lines and the lightning bolt with the 1 at the bottom will execute one line.



gtid	department_name	course_number
903172847	CS	1301
903172847	ISYE	2027
903174238	CS	2316
NULL	NULL	NULL

What is a database?

Databases are simply organized collections of data. Relational databases are a special form of databases that relies on the use of tables to intuitively structure data and to show relationships between different data points. For example, a table within a relational database might look like the following:

Students Table

first_name	last_name	gtid	birthday
Sandy	Cheeks	903172759	1991-09-28
Patrick	Star	903172847	1998-09-28
Spongebob	Squarepants	903174238	1997-09-28

In this example, the column names are first_name, last_name, gtid, and birthday. Each row is called a record, and each column is called an attribute. The table as a whole is called a relation. A database is a collection of tables defined by a schema.

Building a database

In order to build a database, we first need to create a schema which acts as a blueprint for the database. A schema for a database is similar to a class for an object. Recall that a class defines the attributes and behaviors of an object; schemas are similar in that they define the attributes and relationships that each table will have in a database. Schemas define the tabular data that specifies the name of the columns, the datatypes of each column, any constraints on the data, and any relationships between what can be inserted into the table. The schema specifies what data is expected to be inserted into the table, so that all of the data within the database will be consistent and follow logical relationships.

Consider the following example, where we create a university database with information about students at Georgia Tech. Within this database, we will create tables to hold information about students at GT. When we're creating a schema, we typically name the file `<database_name>_schema.sql`.

university_schema.sql

```
DROP DATABASE IF EXISTS university;  
CREATE DATABASE university;  
USE university;
```

* Throughout this handout, all SQL keywords will be capitalized by convention; this is not required

At the top of the schema file, we always start with these three lines, which deletes the database in case it already exists, creates the database again from scratch, and then selects the database to perform SQL operations upon. This resets our database so that we can recreate the database from scratch. If we instead wanted to modify or query from an existing database, we would only include `USE <database_name>` at the top of the file. Now that the database is created, we will continue to build our schema so that our database has tables to store the data.

Creating Tables

In order to populate our database with information, we first need to create a table within our schema. To do so, we use the following syntax:

```
CREATE TABLE <table_name> (  
    <column_name> <column_type> <column_constraints>,  
    [...],  
    PRIMARY KEY (<column_name>),  
    FOREIGN KEY (<column_name>) REFERENCES <table_name_2>(<column_name_2>  
    [...]  
);
```

* SQL convention dictates that column and table names are lowercase and spaces are denoted with underscores

The `<column_type>` for each attribute is split into three categories, each containing similar data types.

- Numerical: int, float, double, decimal
- String: char, varchar, text, enum, string
- Temporal: date, datetime, timestamp, time

Column Modifiers

A column modifier is a word or phrase that modifies a specific column in a table. These are some common `<column_modifiers>` used in MySQL:

- NOT NULL keyword indicates that every value for that column cannot be null.

- UNIQUE indicates that each row within the table should have a unique value for that particular column. In other words, our schema will not allow duplicates for that particular column.
- AUTO_INCREMENT will generate a unique value starting at 1 for the first row, and for each new row, the value will be incremented by 1.

Primary Key

PRIMARY KEY indicates that the column is a unique identifier for each row. In other words, each row within the table should have at least one attribute that allows a user to uniquely identify that row. For example, in the previously illustrated students table, the column GTID can serve as the primary key, because each student should have a uniquely identifiable GTID. On the other hand, first_name would not be a good primary key, because many students may have the same first name. Primary keys are therefore implicitly NOT NULL and UNIQUE, because they must be included and unique for each row that is inserted into the table.

The following is an example implementation of the students table shown above that contains the names, GTIDs, and birthdays of each student enrolled at Georgia Tech. Notice that the table includes the data types and *logical* constraints.

```
CREATE TABLE students (  
    first_name VARCHAR(64) NOT NULL,  
    last_name VARCHAR(64) NOT NULL,  
    gtid CHAR(9) PRIMARY KEY,  
    birthday DATE  
);
```

You may be thinking, shouldn't the birthday attribute have a NOT NULL constraint? Every person has a birthday, right? This is an important question worth asking, and the answer is that it depends. Consider an example where a student fills out a form and all text boxes are mandatory except birthday which is optional. In this case, it is logical that the birthday attribute can be null thus the table above is sufficient. On the contrary, what if the Registrar's office used the students table to verify every student's identity? In this example, we want to guarantee that all students have a birthday so it can be verified, therefore birthday would be NOT NULL.

Foreign Keys

In order to define relationships between one or more tables, we use foreign keys to connect the attributes of the tables. For example, let's say we have another table within the same university_schema.sql file that stores the courses that students are enrolled in:

Enrolled Table

gtid	department_name	course_number
903172847	CS	1301
903172847	ISYE	2027
903174238	CS	2316

```
CREATE TABLE enrolled (
  gtid CHAR(9),
  department_name VARCHAR(4) NOT NULL,
  course_number VARCHAR(4) NOT NULL,
  FOREIGN KEY (gtid) REFERENCES students(gtid)
);
```

It is logical to assume that every student in the enrolled table must also be a student at the university. Therefore, we should establish a relationship between the enrolled and students table such that every GTID in the enrolled table is also in the students table. We can do this by creating a foreign key, which can be implemented within the table constraints of the syntax outlined above.

In order to create a foreign key in the table constraints, the syntax is as follows:

FOREIGN KEY (<column_name1>) REFERENCES <table_name_2>(<column_name2>)

In our example, the enrolled table contains the line:

FOREIGN KEY (gtid) REFERENCES students(gtid)

With the line above added to the schema, we successfully constrained GTID values of the enrolled table to be a subset of the GTID in the students table. In other words, any GTID in the enrolled table must also be a GTID in the birthdays table. This foreign key relationship is formally known as relational integrity, which is a topic discussed further in CS4400.

Another thing to note about foreign keys, is that a foreign key must reference a primary key/unique column. In the example above, the enrolled table references the primary key of the students table, therefore the foreign key constraint is valid. Otherwise, if the enrolled table did not reference the primary key of the students table, then the foreign key constraint would be invalid.

Finally, for the foreign key constraint to be valid, the table whose primary key is being referenced should come before the table whose foreign key is referencing it. It is intuitive since you will want to first create the table before referencing its primary key later on in the schema. In our example,

notice that in `university_schema.sql`, we first create the `students` table before creating the `enrolled` table.

Table Constraints

You have already seen how to use table constraints to construct foreign key relations in the previous section. You can also specify the primary key of a table using table constraints rather than column constraints. The syntax is as follows:

`PRIMARY KEY (<column_name>)`

Based on the new syntax, the implementation for `students` table is the following:

```
CREATE TABLE students (  
    first_name VARCHAR(64) NOT NULL,  
    last_name VARCHAR(64) NOT NULL,  
    gtid CHAR(9),  
    birthday DATE,  
    PRIMARY KEY (gtid)  
);
```

It's up to your discretion to specify the primary keys within the column constraints or within the table constraints. Both options are interchangeable.

However, you must specify the foreign keys within the table constraints.

Notice that the `enrolled` table does not have a primary key. What does this mean? This means that there can be duplicate rows in the `enrolled` table, which is interpreted as a student enrolled more than once for the same course. This problem can be fixed by creating a composite primary key, which can be implemented by adding the following table constraint to the `enrolled` table in our schema:

`PRIMARY KEY (gtid, department_name, course_number)`

Now the `CREATE TABLE` statement of the `enrolled` table becomes the following:

```
CREATE TABLE enrolled (  
    gtid CHAR(9),  
    department_name VARCHAR(4) NOT NULL,  
    course_number VARCHAR(4) NOT NULL,  
    PRIMARY KEY (gtid, department_name, course_number),  
    FOREIGN KEY (gtid) REFERENCES students(gtid)  
);
```

Modifying a Database

Now that we've created a database, we can now exploit its main purpose, which is to store information that can be manipulated or retrieved for future use. Using MySQL, we can manipulate data within tables by either inserting, updating, or deleting particular values.

Insert

After a table is created, it is natural to then insert data that the table was designed to hold. To insert data, we can utilize the MySQL insert command, which is used to add new data into a table. The syntax is as follows:

```
INSERT INTO <table_name> (<column_one>,<column_two>, ...) VALUES (<value_one>,  
<value_two>, ...);
```

For example, if we were to insert a new student into our Enrolled table who's GTID is 903184747 and is enrolled in CS 2316, we would write the following command:

```
INSERT INTO enrolled (gtid, department_name, course_number) VALUES ('903184747', 'CS',  
'2316');
```

Alternatively, if we plan on including a value for each column, we can omit the column names in parentheses:

```
INSERT INTO enrolled VALUES ('903184747', 'CS', '2316');
```

Update

Once data is inserted into a table, it is sometimes desirable to update one or more of the values in a table. The update command is used to modify existing data within a table in a database. The syntax is as follows:

```
UPDATE <table_name> SET <column_name> = <new_value> WHERE <condition>;
```

The WHERE clause is used to specify a condition that must be met in order to update the value. For example, if the student with GTID 903172847 switched from CS1301 to CS1331, we would want to update the row within our enrolled table that has GTID = 903172847 and course_number = 1301, and set course_number to 1331. To accomplish this we would write:

```
UPDATE enrolled SET course_number = '1331' WHERE gtid = '903172847' AND course_number = '1301';
```

Delete

As you might expect, deleting elements from a table is something that many applications require. For example, if a student drops a course then their GTID should be removed from the enrolled table. The delete command is used to remove rows of existing data from a table. The syntax is as follows:

```
DELETE FROM <table_name> WHERE <expression>;
```

Implementing our previous example, if a student with GTID = 903172847 dropped ISYE 2027, we would remove the respective row from the enrolled table using the following:

```
DELETE FROM enrolled WHERE gtid = '903172847' AND course_number = '2027';
```

Basic Queries

The most basic query in SQL outputs certain columns from a given table, and has the syntax:

```
SELECT <column_names> FROM <table_name> WHERE <expression>;
```

SELECT

Select is the fundamental command of all SQL queries that specifies which information a user desires to retrieve from one or more tables. All queries in SQL start with select and output a result set that takes the form of a table. For example, the query: `SELECT * FROM enrolled;` returns all of the columns and rows within the enrolled table.

Round

In MySQL, the `ROUND()` function is used to round a numeric value to a specified number of decimal places or to the nearest integer. Here's a brief explanation of how to use it:

```
ROUND(number, decimals)
```

- ``number``: The numeric value you want to round.
- ``decimals``: The number of decimal places to which you want to round the ``number``. If omitted, it defaults to 0, rounding to the nearest integer.

Example 1: Rounding to an integer

```
SELECT ROUND(5.78); -- Output: 6
```

Example 2: Rounding to a specific number of decimal places

```
SELECT ROUND(5.786, 2); -- Output: 5.79
```

In both examples, ``ROUND()`` returns the rounded value according to the specified number of decimal places or to the nearest integer.

WHERE

WHERE is similar to an if-statement in python. It specifies a certain condition that must be met for a value to be returned by a SELECT statement.

For example, the query: `SELECT * FROM enrolled WHERE gtid = '903172847'`; returns all rows and columns within the enrolled table that have a gtid value of 903172847.

LIKE

LIKE is used with WHERE statements to match a string condition. The `%` sign can be used as a wildcard, i.e. matching any character sequence, similar to the `r"."` regular expression. The `_` character is used to match any character, similar to the `"."` meta character for regular expressions.

The syntax for a LIKE statement is as follows:

```
SELECT <column_names> FROM <table_name> WHERE <column_name> LIKE <expression>;
```

An example of using like with the `%` sign is if we wanted to retrieve all the students who are taking a 2000 level class, we can use the following query:

```
SELECT * FROM enrolled WHERE course_number LIKE '2%';
```

An example of using an `_` is if we wanted to retrieve all the students who are either ISYE 2027 or ISYE 2028

```
SELECT * FROM enrolled WHERE course_number LIKE '202_';
```

If we instead replaced the `_` with a `%` sign we would match 2027 and 2028 however the `%` sign would match 202 and 20278 while the `_` would not.

ORDER BY

ORDER BY specifies the columns by which to sort the result of a query in either ascending or descending order, with the default being ascending. The syntax is:

```
SELECT <column_names> FROM <table_name> ORDER BY <column_names> ASC|DESC;
```

For example, to order our students in the enrolled table by their GTID, we would perform the following query:

```
SELECT * FROM enrolled ORDER BY gtid ASC;
```

LIMIT

The LIMIT keyword is used to restrict the number of rows returned from a query. The syntax is as follows:

```
SELECT <column_names> FROM <table_name> LIMIT <limit_number>;
```

For example, if we only wanted to return the first student from the enrolled table, ordered by GTID, we would perform the following query:

```
SELECT * FROM enrolled ORDER BY gtid ASC LIMIT 1;
```

OFFSET

The OFFSET keyword skips the offset rows before beginning to return any rows.

```
SELECT <column_names> FROM <table_name> LIMIT <limit_number> OFFSET <offset_num>;
```

For example, if we wanted to return the 2nd and 3rd place winners in a race....

```
SELECT * FROM race_times ORDER BY time ASC LIMIT 2 OFFSET 1;
```


DISTINCT

The DISTINCT keyword retrieves the unique combination of the column names requested from a given table. The syntax is as follows:

```
SELECT DISTINCT <column_names> FROM <table_name>;
```

For example, if we only wanted to return the unique department names from the enrolled table we would use the following:

```
SELECT DISTINCT department_name FROM enrolled;
```

Another example, is if we wanted to return the unique combinations of gtids and department names from the enrolled table we would use the following:

```
SELECT DISTINCT gtid, department_name FROM enrolled;
```

GROUP BY

GROUP BY is used in a select statement to collapse all records that have the same column value into a single group. The grouped sets are then used to apply aggregate functions such as sum, count, max, min, and avg. The syntax is as follows:

```
SELECT <column_names>, <aggregate_function>(<column_name>) FROM <table_name> GROUP BY <column_name>;
```

For example, if we were to count the number of classes each student was enrolled in, we could perform the following query:

```
SELECT gtid, count(*) FROM enrolled GROUP BY gtid;
```

HAVING

HAVING filters specific rows of the grouped result set. It is similar to WHERE in that it is used to filter data, however there are some differences. HAVING can be used with an aggregate while WHERE cannot and WHERE precedes GROUP BY while HAVING follows GROUP BY. The examples below will discuss this further. The general syntax for using HAVING with GROUP BY is as follows:

```
SELECT <column_names> FROM <table_name> GROUP BY <column_name> HAVING <expression>;
```

If we were to perform the query from the previous section, but we wanted to only count the courses that are in the CS department, then we could perform the following query:

```
SELECT gtid, count(*) FROM enrolled WHERE department_name = 'CS' GROUP BY gtid;
```

Which filters the data before the GROUP BY clause using WHERE. In this case, we cannot use HAVING to filter the departments, instead we must use WHERE before the GROUP BY clause.

Now to illustrate that aggregates can be used with HAVING, consider the following example where we select all the students from the enrolled table that are taking 2 or more classes.

```
SELECT gtid, count(*) FROM enrolled GROUP BY gtid HAVING count(*) >= 2;
```

The query above is valid using HAVING. However, if we replace HAVING with WHERE, the query is invalid. The main difference between the WHERE and HAVING clauses comes when used together with GROUP BY. In that case WHERE is used to filter rows before grouping and HAVING is used to filter rows after grouping.

JOIN

The FROM clause in the query accepts one or more tables from the database and can combine them into one table using the JOIN operation. The three types of joins that we will discuss in this class are CROSS JOIN, INNER JOIN, OUTER JOIN, and NATURAL JOIN.

CROSS JOIN

A CROSS JOIN matches every row of the first table with every row of the second table. It is equivalent to a cartesian product (or cross product) of two tables. The syntax for a CROSS JOIN is:

```
SELECT <column_names> FROM <table1> CROSS JOIN <table2>;  
or  
SELECT <column_names> FROM <table1>, <table2>;
```

For example, if we were to cross join the previously defined enrolled and students tables, we would receive the resulting table:

```
SELECT * FROM students CROSS JOIN enrolled;
```

first_name	last_name	gtid	birthday	gtid	department_name	course_number
Sandy	Cheeks	903172759	1991-09-28	903172847	CS	1301
Patrick	Star	903172847	1998-09-28	903172847	CS	1301
Spongebob	Squarepants	903174238	1997-09-28	903172847	CS	1301
Sandy	Cheeks	903172759	1991-09-28	903172847	ISYE	2027
Patrick	Star	903172847	1998-09-28	903172847	ISYE	2027
Spongebob	Squarepants	903174238	1997-09-28	903172847	ISYE	2027
Sandy	Cheeks	903172759	1991-09-28	903174238	CS	2316
Patrick	Star	903172847	1998-09-28	903174238	CS	2316
Spongebob	Squarepants	903174238	1997-09-28	903174238	CS	2316

INNER JOIN

An INNER JOIN matches rows across two tables given an ON condition. The general format of an INNER JOIN is as follows:

```
SELECT <column_names> FROM <table1> INNER JOIN <table2> ON <condition>;
or
SELECT <column_names> FROM <table1> JOIN <table2> ON <condition>;
```

In our case, using the enrolled and birthdays tables, it makes sense to join the two tables on their gtid column:

```
SELECT * FROM students JOIN enrolled ON students.gtid = enrolled.gtid;
Or equivalently
SELECT * FROM students INNER JOIN enrolled ON students.gtid = enrolled.gtid;
```

first_name	last_name	gtid	birthday	gtid	department_name	course_number
Patrick	Star	903172847	1998-09-28	903172847	CS	1301
Patrick	Star	903172847	1998-09-28	903172847	ISYE	2027
Spongebob	Squarepants	903174238	1997-09-28	903174238	CS	2316

OUTER JOINS

An OUTER JOIN is similar to an INNER JOIN in that it matches rows across two tables given an ON condition and retrieving the records where the rows share matching values for the column given in the ON condition. The OUTER JOIN, however, also retrieves all the rows in the left table (in the LEFT JOIN) or the right table (in the RIGHT JOIN) whether they match or not.

Using our enrolled and birthdays tables and joining the two tables on their gtid column, the following shows what would be retrieved by a left outer join, employing the keywords LEFT JOIN, and a right outer join employing the keywords RIGHT JOIN.

SELECT * FROM students LEFT JOIN enrolled ON students.gtid = enrolled.gtid;

first_name	last_name	gtid	birthday	gtid	department_name	course_number
Sandy	Cheeks	903172759	1991-09-28	NULL	NULL	NULL
Patrick	Star	903172847	1998-09-28	903172847	CS	1301
Patrick	Star	903172847	1998-09-28	903172847	ISYE	2027
Spongebob	Squarepants	903174238	1997-09-28	903174238	CS	2316

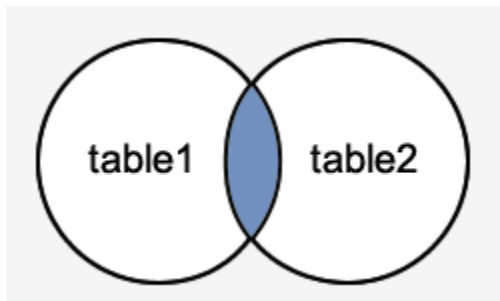
SELECT * FROM students RIGHT JOIN enrolled ON students.gtid = enrolled.gtid;

first_name	last_name	gtid	birthday	gtid	department_name	course_number
Patrick	Star	903172847	1998-09-28	903172847	CS	1301
Patrick	Star	903172847	1998-09-28	903172847	ISYE	2027
Spongebob	Squarepants	903174238	1997-09-28	903174238	CS	2316

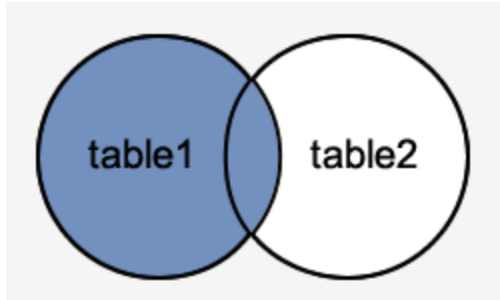
The LEFT OUTER JOIN shows an additional row not shown in the RIGHT OUTER JOIN nor the INNER JOIN result sets. This additional row is caused by the Sandy Cheeks value not matching on gtid but still being included. The RIGHT OUTER JOIN results are the same as the INNER JOIN results because all rows in that table were already included in the resulting set of rows.

The visuals below can help demonstrate the difference between the INNER JOIN, the LEFT OUTER JOIN, and the RIGHT OUTER JOIN.

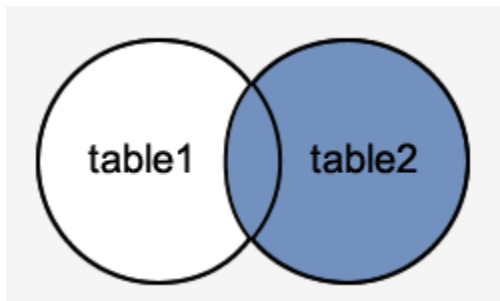
INNER JOIN



LEFT OUTER JOIN



RIGHT OUTER JOIN



NATURAL JOIN

A NATURAL JOIN is similar to an inner join in that it is combining two tables based on common values in a given column. However, the common column must have the same name across both tables, and the resulting table will not repeat the column name as it did in the inner join.

`SELECT <column_names> from <table1> NATURAL JOIN <table2>;`

Or equivalently

`SELECT <column_names> from <table1> JOIN <table2> USING(similar_column);`

In our example, we would combine on GTID and obtain the following table:

`SELECT * FROM students JOIN enrolled USING (gtid);`

gtid	first_name	last_name	birthday	department_name	course_number
903172847	Patrick	Star	1998-09-28	CS	1301
903172847	Patrick	Star	1998-09-28	ISYE	2027
903174238	Spongebob	Squarepants	1997-09-28	CS	2316

Nested Queries

A nested query is simply a query within another query. In general, you have a SELECT statement that needs the results from another SELECT statement to determine the full answer to a specific question. The general format following this pattern:

```
SELECT <column_names> FROM <table_name>  
WHERE <column_name> <operator> (SELECT <column_names> FROM <table_name>  
WHERE <expression>);
```

Here is an example of a nested query that involves two tables.

```
SELECT * FROM students WHERE gtid in  
    (SELECT gtid FROM enrolled where course_number = "2316");
```

You may notice from the example above that nested queries can be used to combine multiple tables similar to the way that inner joins work, but they can also be used when only one table is involved as in the following example.

Here is an example of a nested query that needs information from the same table.

```
SELECT first_name,last_name, birthday FROM students WHERE gtid <  
    (SELECT avg(gtid) FROM students);
```

You can put a nested SELECT within the WHERE clause with comparison operators (<, >, =, !=) or the IN, NOT IN, ANY, or ALL operators. The latter group of operators are used when the result of the nested query is a set of results rather than a single value.

WITH and AS

In SQL aliases are used to give a table, columns from a table, or SELECT statement results a temporary name. The alias only lasts for the duration for the one particular query you are creating. They are used to make complex queries more readable, or to prevent ambiguity when the same table or column is used in multiple portions of a nested query.

Here is an example of using AS to shorten long column names:

```
SELECT CustomerID AS ID, CustomerName AS Customer FROM Customers;
```

Here is an example of using AS to refer to multiple columns:

```
SELECT CustomerName, CONCAT(Address,', ',PostalCode,', ',City,',',Country) AS Address FROM  
Customers;
```

The optional SQL keyword WITH goes along with AS to rename SELECT statements to make your queries more readable and avoid ambiguity when developing complex queries. The general syntax is shown below.

```
WITH <query_name> AS
    (SELECT <column_names> FROM <table_name> WHERE <expression>)
    SELECT <column_names> FROM <query_name> WHERE <expression>
```

Here is an example of using WITH and AS rename the select statement results from a nested query:

```
SELECT CustomerName, CONCAT(Address,' ',PostalCode,' ',City,',',Country) AS Address FROM
Customers;
```

Union

In MySQL, the UNION operator is used to combine the result sets of two or more SELECT statements. Here's a basic syntax:

```
SELECT column1, column2, ...
FROM table1
WHERE condition
UNION
SELECT column1, column2, ...
FROM table2
WHERE condition;
```

Here's a breakdown:

The SELECT statements within the UNION must have the same number of columns in the result sets with similar data types.

The WHERE clauses are optional, and you can adjust them based on your specific requirements.

For example:

```
SELECT employee_id, employee_name
FROM employees
WHERE department = 'Sales'
UNION
SELECT employee_id, employee_name
```

```
FROM temp_employees  
WHERE department = 'Sales';
```

This query retrieves the `employee_id` and `employee_name` from both the `employees` and `temp_employees` tables for employees in the 'Sales' department.

Remember, if you want to include duplicate rows, you can use `UNION ALL` instead of `UNION`.